Name: Kashish Paryani Batch: T21 Roll No: 83

EXPERIMENT 2

Aim: To perform Version Control using GIT.

Git is a free and open source distributed version control system designed to handle everything from small to very large projects with speed and efficiency. Git is easy to learn and has a tiny footprint with lightning fast performance. It outclasses SCM tools like Subversion, CVS, Perforce, and ClearCase with features like cheap local branching, convenient staging areas, and multiple workflows.

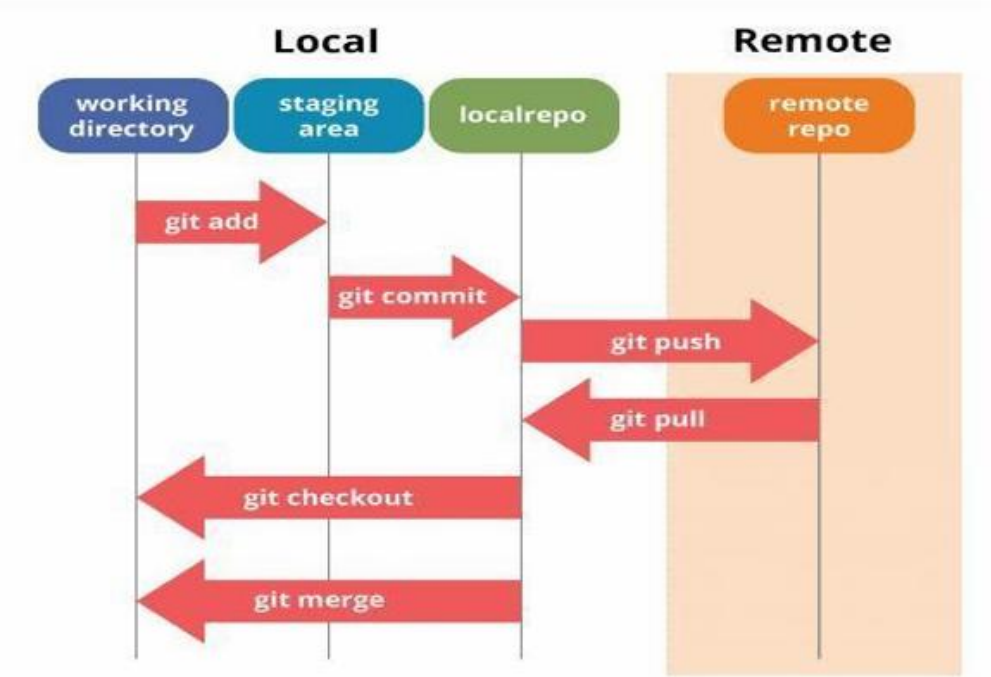
Some of the basic operations in Git are:

1. Initialize
2. Add
3. Commit
4. Pull
5. Push

Some advanced Git operations are:

1. Branching
2. Merging
3. Rebasing

The following diagram depicts all the supported operations in GIT:



Installation of GIT

1) In windows, download GIT from https://git-scm.com/ and perform the straightforward installation.

The output of GIT Bash in windows shell is as below:

```
Microsoft Windows [Version 10.0.26200.8246]
(c) Microsoft Corporation. All rights reserved.

C:\Users\kashishparyani>git --version
git version 2.53.0.windows.3
```

To perform version control, let us create a directory dvcs (Distributed version control system) and change directory to dvcs.

$ mkdir git-dvcs
$ cd git-dvcs/

Now check the user information using

$ git config-–global
As there are no users defined, let us define it using following two commands

$ git config--global user.name "deepa"
$ git config--global user.email "tsecpractical@gmail.com"

Now, check the list of users

$ git config--global--list
user.name=bhushan
user.email=bhushan.jadhav1@gmail.com

```
C:\Users\kashishparyani>git config --global user.name "Kashish"

C:\Users\kashishparyani>git config --global user.email "kashishh.paryanii@gmail.com"

C:\Users\kashishparyani>git config --global --list
user.name=Kashish
user.email=kashishh.paryanii@gmail.com
```

Let us create a repository for version control named “git-demo-project”
$ mkdir git-demo-project
$ cd git-demo-project/
Now, initialize the repository using following command
$ git init
The output of above command shown below which adds .git hidden directory in current

repository.

```
C:\Users\kashishparyani>mkdir git-dvcs

C:\Users\kashishparyani>cd git-dvcs

C:\Users\kashishparyani\git-dvcs>git init
Initialized empty Git repository in C:/Users/kashishparyani/git-dvcs/.git/
```

Now, let us add some files inside our repository “git-demo-project”
To add files in the repository by create or copy some doc,html,image files inside current directory to see index and staging area. The add command is used along with dot (. Dot means current directory) for adding files in current repository i.e. making them in staging mode. They are untracked until we commit them.
$ git add .
Index and staging area

To check the status of repository, use
$ git status
Which will show you some untrack files, so untracks files can be tracked using commit command.

Now, let us commit the changes
$ git commit-m "First Commit" (#here-m for message)

```
C:\Users\kashishparyani\git-dvcs\git-demo-project>git add .

C:\Users\kashishparyani\git-dvcs\git-demo-project>git status
On branch master

No commits yet

nothing to commit (create/copy files and use "git add" to track)

C:\Users\kashishparyani\git-dvcs\git-demo-project>git commit -m "First Commit"
On branch master

Initial commit

nothing to commit (create/copy files and use "git add" to track)

C:\Users\kashishparyani\git-dvcs\git-demo-project>
```

Add index.html in our directory
$ git add .
$ git commit-am "express Commit" (#Here-a used for express commit)
$ nano index.html

```
C:\Users\kashishparyani>cd git-dvcs

C:\Users\kashishparyani\git-dvcs>cd git-demo-project

C:\Users\kashishparyani\git-dvcs\git-demo-project>git init
Reinitialized existing Git repository in C:/Users/kashishparyani/git-dvcs/git-demo-project/.git/

C:\Users\kashishparyani\git-dvcs\git-demo-project>git add index.html

C:\Users\kashishparyani\git-dvcs\git-demo-project>touch teststatus
'touch' is not recognized as an internal or external command,
operable program or batch file.

C:\Users\kashishparyani\git-dvcs\git-demo-project>git checkout -- index.html

C:\Users\kashishparyani\git-dvcs\git-demo-project>git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   index.html
```

Changes are Discarded by checkout

(use "git add ..." to update what will be committed)

(use "git restore ..." to discard changes in working directory)

$ git add index.html

$ git commit-am "Express commit"

```
C:\Users\kashishparyani\git-dvcs\git-demo-project>git commit -am "Express Commit"
[master (root-commit) 565060e] Express Commit
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 index.html

C:\Users\kashishparyani\git-dvcs\git-demo-project>git status
On branch master
nothing to commit, working tree clean

C:\Users\kashishparyani\git-dvcs\git-demo-project>
```

Now let us see history of commits. The log command is used for seeing the commit history.

$ git log

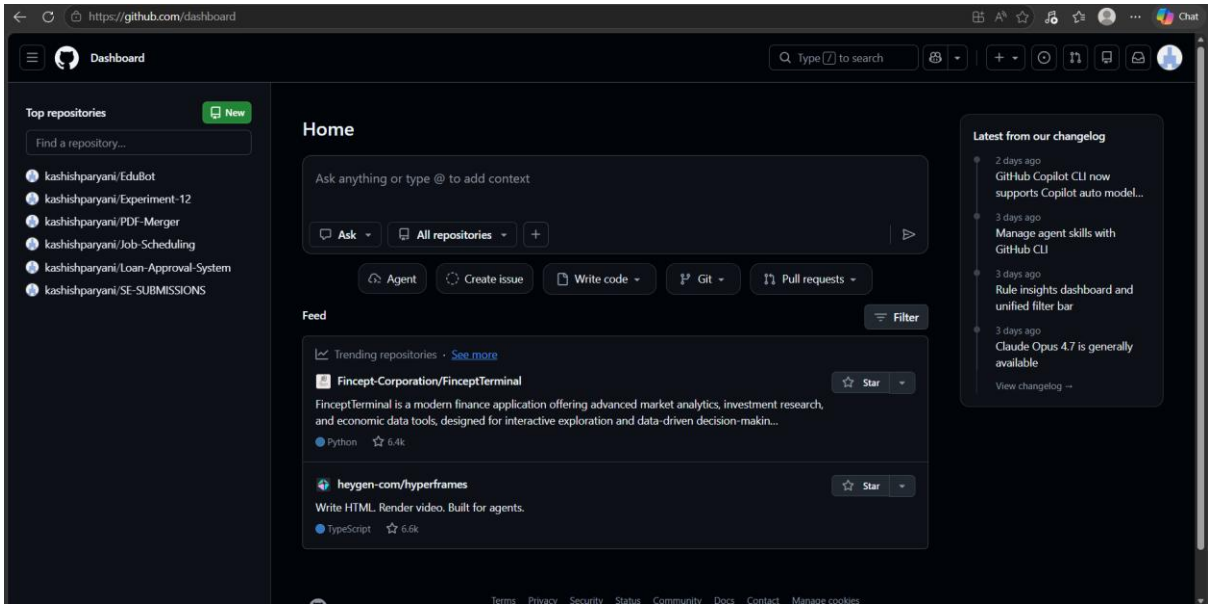
To see all the operation in oneline use the –-oneline option in log command

```
C:\Users\kashishparyani\git-dvcs\git-demo-project>git log
commit 565060ebac64e283c9c0342fe39e7450af431ebe (HEAD -> master)
Author: Kashish <kashishh.paryanii@gmail.com>
Date:   Mon Apr 20 04:53:55 2026 +0530

    Express Commit

C:\Users\kashishparyani\git-dvcs\git-demo-project>git log --oneline
565060e (HEAD -> master) Express Commit

C:\Users\kashishparyani\git-dvcs\git-demo-project>git log --oneline index.html
565060e (HEAD -> master) Express Commit

C:\Users\kashishparyani\git-dvcs\git-demo-project>
```

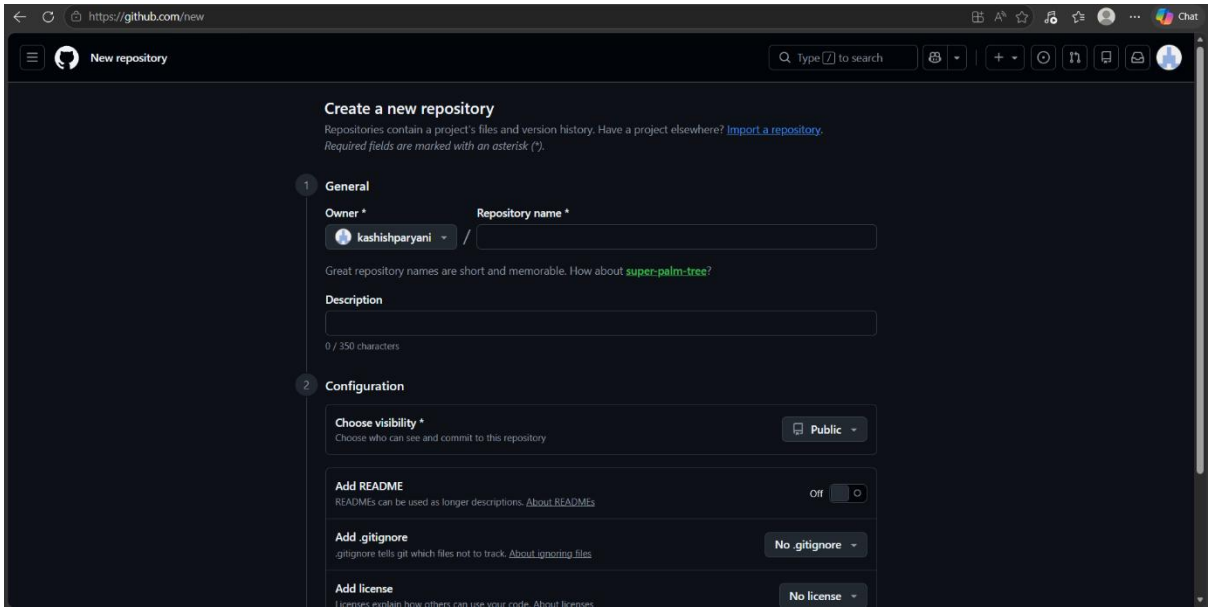
Example 2: Performing Version control in GITHUB with Pull and Push commands

First open Github.com and create a new account. After verifying account through E-mail, create a Repository on github.com.

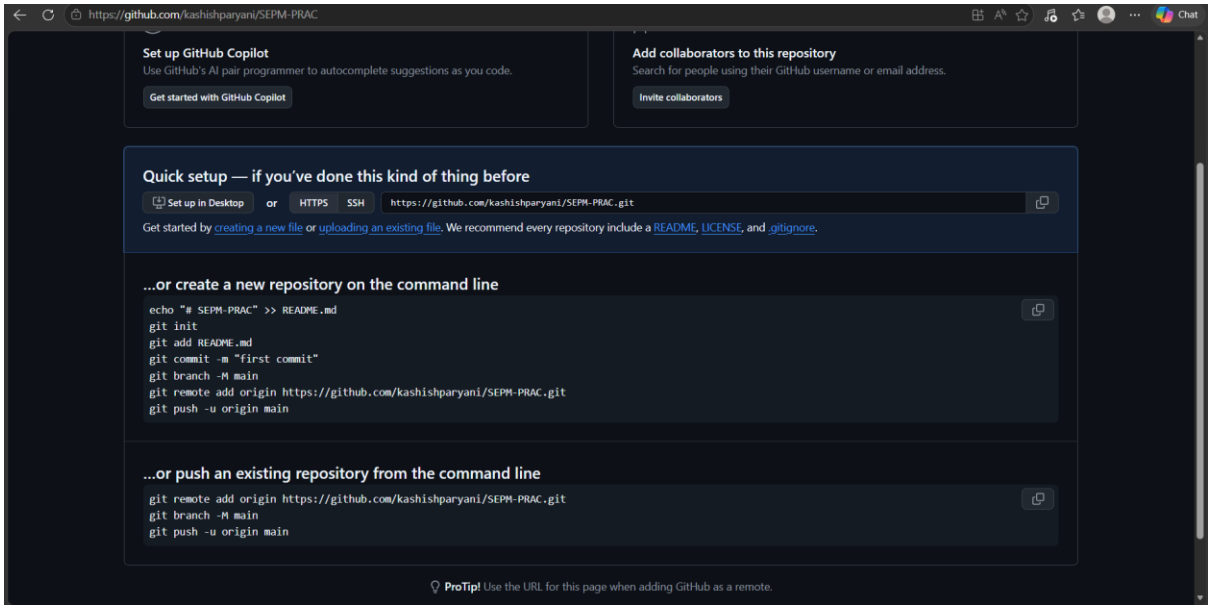
Open github.com create an account



After login Select New repository from the menu.



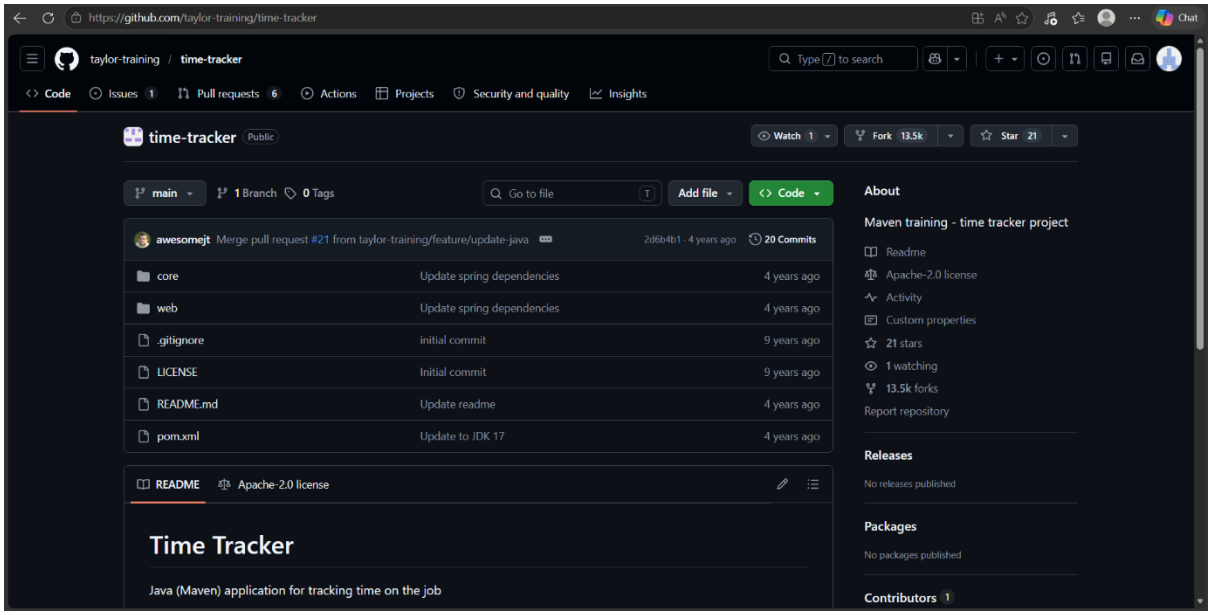
Now Specify a Name to repository and select public option followed by create repository

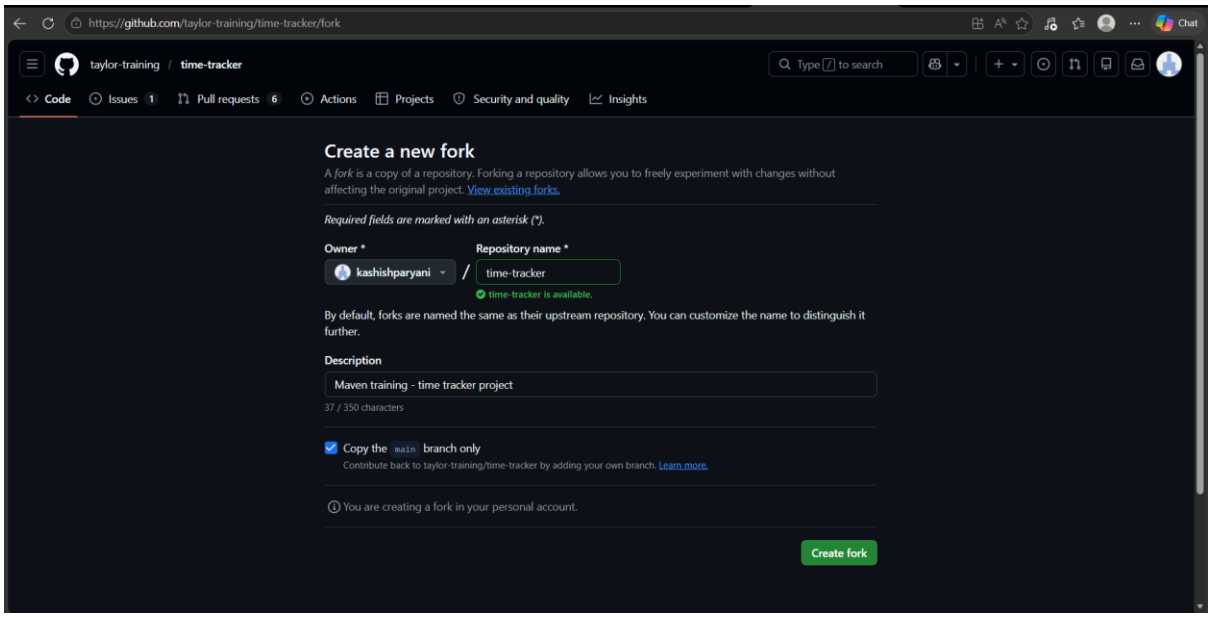


By default, we can create public repository in Github. So we can copy the entire public repository of any other users in to own account using "FORK" Operation. Now fork the repository (Sharing with other users who wants to contribute).

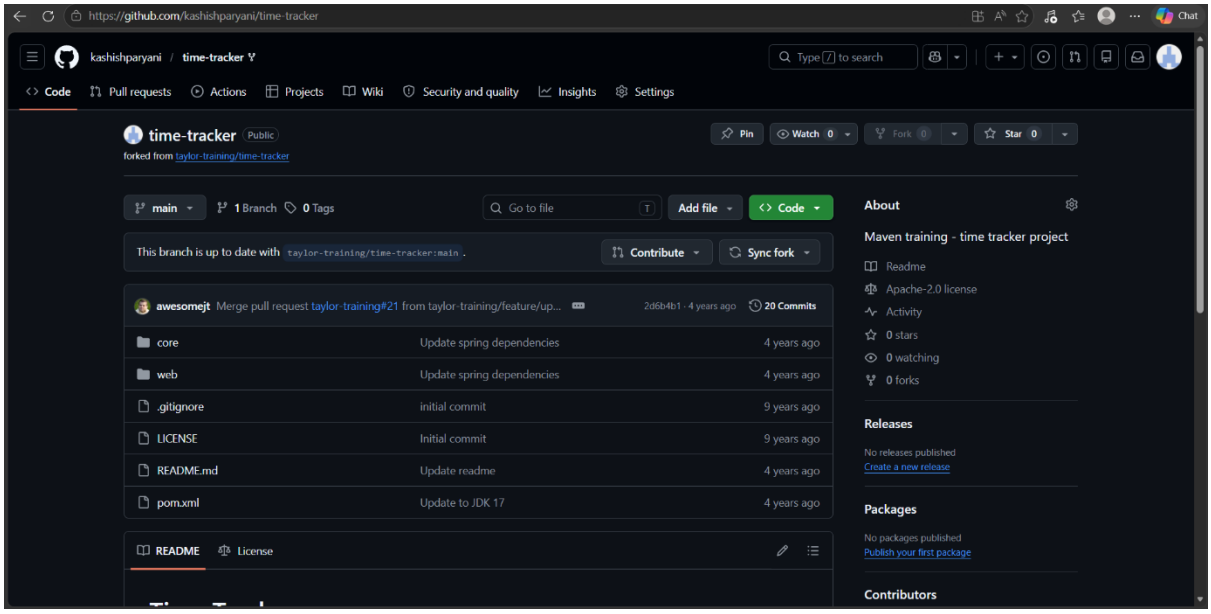
Login with another account Copy and Paste URL of repository then just click on fork to clone to others account.
Suppose we want to fork public repository "timetracker". So search for "timetracker" github repository on google and once its opened clicked on "Fork button" from the top of the github web page as shown below.

Name: Kashish Paryani Batch: T21 Roll No: 83

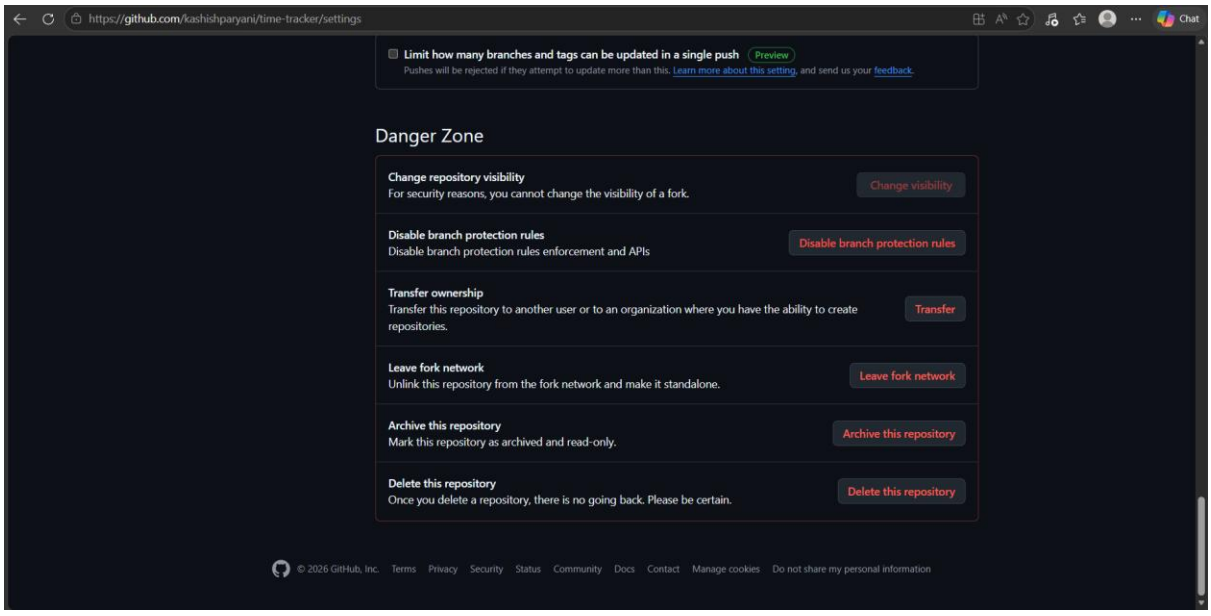




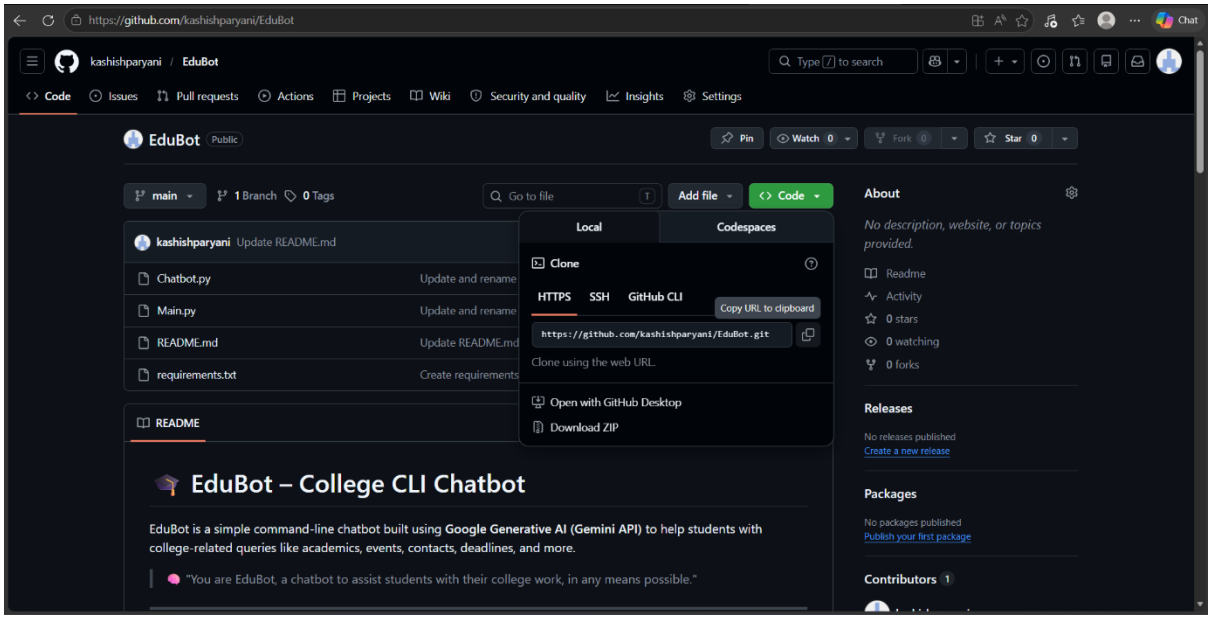
After fork it will be added in your local repository.



To delete the repository, open the desired repository you want to delete and go to the settings option. There you will see delete repository button to delete it.



Now, if you want to download a repository in local machine, then git clone command is used followed by path to repository. In GitHub the path of repository can be known through clone or download button and it can be downloaded using git clone command as shown below.




```
C:\Users\kashishparyani\git-dvcs\git-demo-project>git clone https://github.com/kashishparyani/EduBot.git
Cloning into 'EduBot'...
remote: Enumerating objects: 27, done.
remote: Counting objects: 100% (27/27), done.
remote: Compressing objects: 100% (22/22), done.
remote: Total 27 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (27/27), 10.52 KiB | 5.26 MiB/s, done.
Resolving deltas: 100% (4/4), done.

C:\Users\kashishparyani\git-dvcs\git-demo-project>
```

Pull and Push Processes

The pull command used to fetch the repository from github to local while push is used to commit files from local repository to Github.

Push (): Push changes to Web repository

Pull (): Pull changes to Local repository

The following commands are used for pull and push repositories

A) Push command

```
C:\Users\kashishparyani\git-dvcs\git-demo-project>git remote add origin https://github.com/kashishparyani/SEPM-PRAC.git

C:\Users\kashishparyani\git-dvcs\git-demo-project>git remote show origin
* remote origin
  Fetch URL: https://github.com/kashishparyani/SEPM-PRAC.git
  Push  URL: https://github.com/kashishparyani/SEPM-PRAC.git
  HEAD branch: (unknown)

C:\Users\kashishparyani\git-dvcs\git-demo-project>git commit -m "Initial Commit"
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        EduBot/

nothing added to commit but untracked files present (use "git add" to track)
```

If you add remote again then will show you fatal error.

$ git remote add origin https://github.com/bhushanjadhav1/Myrepository.git

fatal: remote origin already exists.

So, to delete origin rm origin command is used

$ git remote rm origin

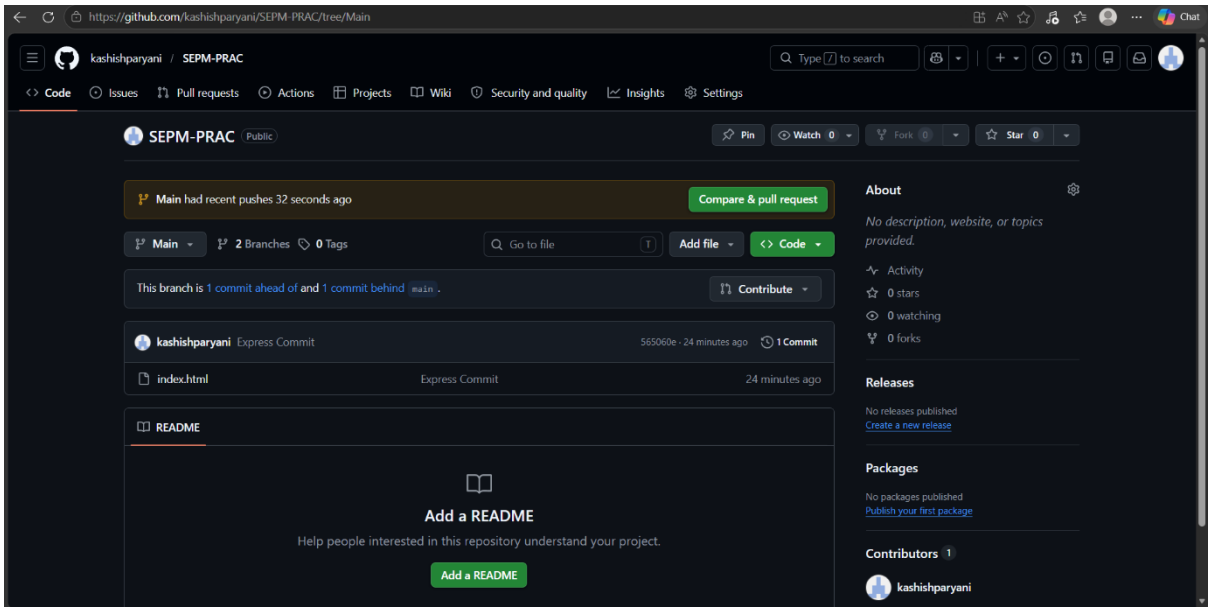
Now, to push the local repository to remote github following command is used

$ git push-u origin master

```
C:\Users\kashishparyani\git-dvcs\git-demo-project>git branch -m Main

C:\Users\kashishparyani\git-dvcs\git-demo-project>git push -u origin main
info: please complete authentication in your browser...
```

```
C:\Users\kashishparyani\git-dvcs\git-demo-project>git push -u origin Main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 221 bytes | 221.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'Main' on GitHub by visiting:
remote:      https://github.com/kashishparyani/SEPM-PRAC/pull/new/Main
remote:
To https://github.com/kashishparyani/SEPM-PRAC.git
 * [new branch]      Main -> Main
branch 'Main' set up to track 'origin/Main'.

C:\Users\kashishparyani\git-dvcs\git-demo-project>
```

Now you can check the github for updated contents.



B) Pull Changes

Pull command is used to download the remote updated repository into local one. The command for download is

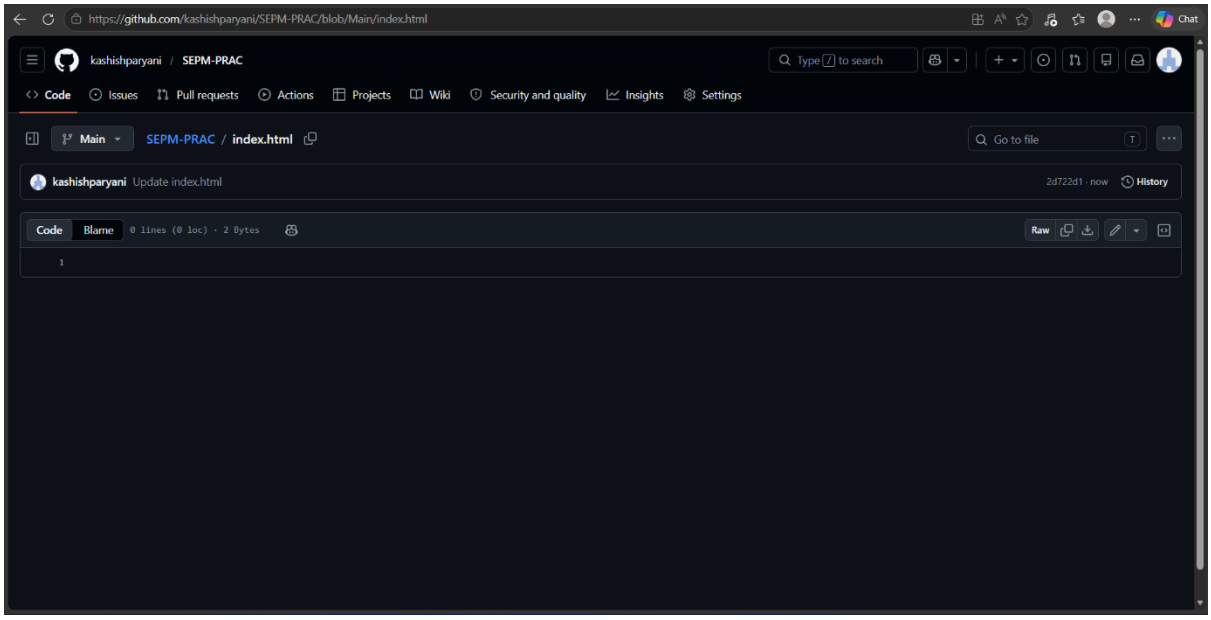
$ git pull

```
C:\Users\kashishparyani\git-dvcs\git-demo-project>git pull
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 852 bytes | 106.00 KiB/s, done.
From https://github.com/kashishparyani/SEPM-PRAC
 * [new branch]      main       -> origin/main
Already up to date.

C:\Users\kashishparyani\git-dvcs\git-demo-project>
```

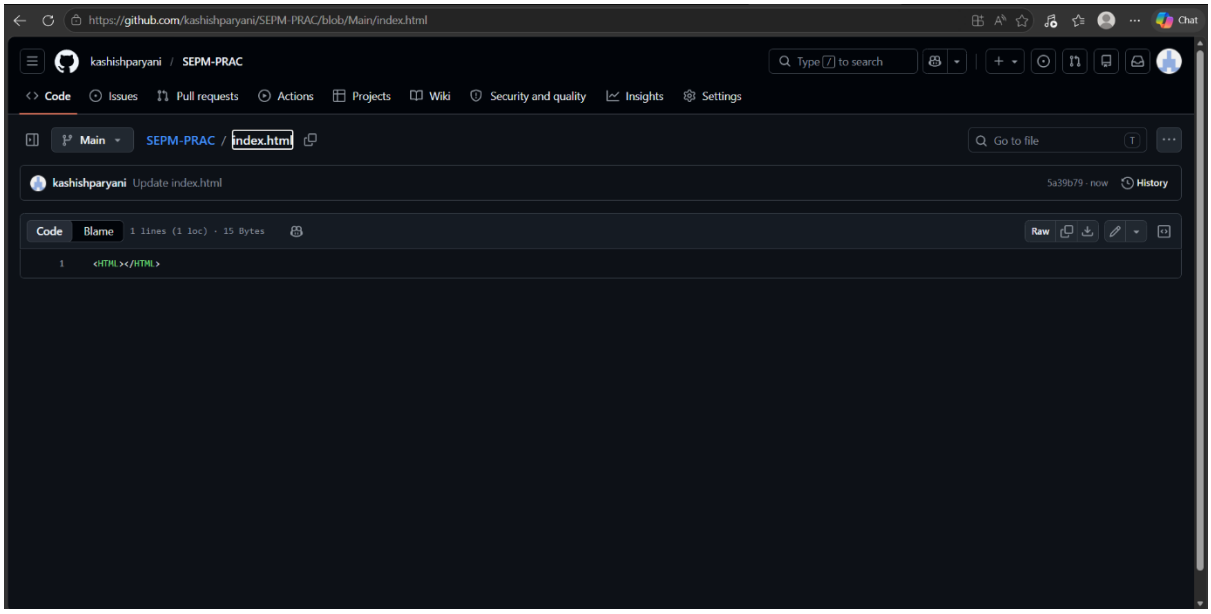
C) Fetch

Suppose you have a file in github and you have changed it.

From:



To:



Now we use fetch command to fetch the changes, which will show you both the files like original and changed in local repository.

$ git fetch

```
C:\Users\kashishparyani\git-dvcs\git-demo-project>git fetch
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 903 bytes | 112.00 KiB/s, done.
From https://github.com/kashishparyani/SEPM-PRAC
 * [new branch]      Main       -> origin/Main

C:\Users\kashishparyani\git-dvcs\git-demo-project>
```

Here fetch will not show you like updated changes file as like push. So, use merge command to merge the changes so use following command for merge.

$ git merge origin/master

```
C:\Users\kashishparyani\git-dvcs\git-demo-project>git merge origin/Main
Updating 565060e..5a39b79
Fast-forward
 index.html | 1 +
 1 file changed, 1 insertion(+)

C:\Users\kashishparyani\git-dvcs\git-demo-project>
```

Online Resource: https://dzone.com/articles/top-20-git-commands-with-examples